

CarSpec AI: Vehicle Multi-Attribute Recognition

Module 2 Project · Computer Vision

1. The Problem

Vehicle attribute recognition sits underneath a lot of practical systems - used-car pricing, fleet inventory, insurance verification. The input is one photo of a car exterior; the output the business actually wants is a small structured record - what type, how many doors, how many seats.

Most existing systems handle this by training one classifier per attribute and treating the rest as someone else's problem. That works, but it leaves information on the table. A coupe almost always has two doors and two seats; an MPV almost always has five doors and seven seats. If the model knows the type is "coupe," it already knows a lot about the door and seat counts. Multi-task learning exists precisely to exploit this kind of correlation.

The second problem is trust. A pure CNN will happily tell you "MPV, 5 doors, 7 seats" and offer no explanation. A used-car buyer who just uploaded a photo of their sedan and got back "MPV" has no way to tell whether the model is right or broken. So the project also tries to surface *why* a prediction was made, by feeding handcrafted visual features (color, shape, texture) alongside the deep features and turning them into plain-language explanations.

What I wanted at the end was one model, three attributes, and an audit trail a human can read.

2. Data

The intended dataset is CompCars [1] - 136,726 whole-vehicle images across 1,716 models, with attributes for type, door count, seat count, max speed, and displacement. It's non-commercial research use, distributed via Dropbox/Google Drive with a password for the archive.

CompCars requires manual download, and the archive is large. To get the pipeline running end-to-end without blocking on the download, the project also ships a synthetic generator (`scripts/synthetic_data.py`) that draws simplified vehicle silhouettes and assigns attributes using the same correlations found in real data (coupes → 2 doors/2 seats, MPVs → 5 doors/7 seats, etc.). The generator had a labeling bug in early versions (attributes were aggregated per `model_id` instead of per image, so ~98% of samples had labels uncorrelated with their pixels); this was fixed before retraining.

For the final evaluation run, I crawled 5,006 real car photos from Bing Image Search using `icrawler` (5 search keywords per class × 400 target images), covering sedan, suv, mpv, coupe, and hatchback. After deduplication (MD5 hash), corrupted-image filtering (PIL verify), and aspect-ratio filtering (0.5–3.0), 4,869 valid images remained. I used an 80/20 stratified split - 3,895 for training, 974 for held-out test. Five images correctly predicted by the deep model with high confidence were selected from the test set as live demo samples and removed from the evaluation set to avoid overlap, leaving 969 test images. The crawler code is in `scripts/download_large_dataset.py` and the cleaning pipeline in `scripts/clean_dataset.py`.

Images are resized to 224×224 and normalized to [0,1] - standard for ImageNet-pretrained backbones. The split is stratified by car type so class proportions stay consistent across train and test.

Attribute mapping collapses raw CompCars labels into five car types, three door counts (2/4/5), and three seat counts (2/5/7).

3. Related Work

The original CompCars paper [1] used GoogLeNet for fine-grained classification and attribute prediction, but with a single-task setup - one model per attribute. Subsequent work moved toward multi-task architectures, where a shared backbone (typically ResNet50) feeds multiple attribute heads, and various attention modules have been layered on top to improve feature selection.

On the interpretability side, Grad-CAM [4] is the standard tool for visualizing where a CNN is looking, and several prior efforts have used handcrafted visual features (color histograms, HOG, texture) as the basis for vehicle-type classifiers. This project borrows the handcrafted-feature idea but goes further by turning the feature values into natural-language statements (“dominant color is red,” “aspect ratio 1.2 □ leans SUV”) rather than just reporting feature importances.

The novelty here isn’t any single component - it’s the combination of multi-task learning with handcrafted features that double as an explanation layer, all packaged in a deployable web app.

4. How Success Is Measured

Accuracy is the primary metric, computed independently for each of the three tasks (car_type, door_count, seat_count). I also track top-5 accuracy, a confidence/coverage curve for selective prediction, per-class accuracy for head/tail analysis, and robustness under image corruptions. The confusion matrix is saved as a plot for every model × task combination.

The evaluation set is 969 held-out real photos (after removing 5 demo samples from the 974-image test split). The same split is used for all three models (naive, classical, deep) so the numbers are directly comparable.

5. Three Models

The naive baseline is a `DummyClassifier` with `strategy="most_frequent"` - the floor any real model has to clear. For car_type (5 classes) the floor is 23% (majority class is “coupe” due to the slight imbalance); for door_count and seat_count (3 classes each, with “5” dominant) it’s 58%.

The classical model is a Random Forest (`RandomForestClassifier`, 100 trees, `max_depth=12`, `class_weight="balanced"`) over 50 handcrafted features:

- HSV color histogram, 24 dims (8 bins × 3 channels)
- Aspect ratio, 1 dim
- Edge density (Sobel), 2 dims
- Body proportion (upper/lower brightness), 3 dims
- Left-right symmetry, 1 dim
- HOG statistics, 3 dims
- LBP texture histogram, 16 dims

These are the same features used in the explanation layer, so the classical model is interpretable by construction - every prediction can be traced back to which feature values triggered which tree splits.

The deep model replaces MobileNetV2's classifier [2] with a 256-dim shared FC layer (ReLU + Dropout 0.3) and three linear heads for `car_type` / `door_count` / `seat_count`. Loss is the sum of three CrossEntropy losses with label smoothing 0.1; optimizer is Adam at $\text{lr}=1\text{e-}3$ with cosine annealing to $1\text{e-}5$ over 80 epochs. The last ten inverted-residual blocks of the backbone (features.10–19) are unfrozen - the rest stays frozen to preserve ImageNet features. Training runs for 80 epochs with early stopping (patience 15) and on-the-fly augmentation (random horizontal flip, color jitter, affine, rotation, random erasing). Training uses Apple MPS (Metal Performance Shaders) on an M1 Pro for GPU acceleration. The full training script is `scripts/train_large.py`.

All three models share the same `fit` / `predict` / `predict_proba` / `save` / `load` interface in `scripts/model.py`, which makes them swappable inside the experiment harness.

6. Data Pipeline

The pipeline has two entry points: `scripts/download_large_dataset.py` (crawl real images) + `scripts/clean_dataset.py` (dedup/filter), then `scripts/train_large.py` (train classical RF + deep MobileNetV2 on the cleaned set) and `scripts/eval_large.py` (re-evaluate saved models and regenerate `metrics.json`). The feature extraction step (`scripts/features.py`) is shared between the classical model's input and the deep model's optional auxiliary input, so the two never see inconsistent feature definitions.

Preprocessing choices are boring on purpose: 224×224 because that's what MobileNetV2 expects, $[0,1]$ normalization because that's what ImageNet pretraining assumes, and a stratified split so the class proportions stay consistent across train and test. No exotic augmentation in the classical path - the handcrafted features are stable to small perturbations.

7. Hyperparameters

Most hyperparameters were chosen by informal grid search rather than a formal sweep, since the training set is small enough that a rigorous search would overfit the validation signal.

For the Random Forest, `n_estimators` was tried at 50/100/200 and settled at 100 - more trees gave diminishing returns, fewer hurt stability. `max_depth=12` was a sweet spot, with deeper trees overfitting the training set and shallower ones underfitting. `class_weight="balanced"` is non-negotiable given the slight class imbalance (coupe 1117 vs suv 894).

For the deep model, the main lever was how much of the backbone to unfreeze. With only the last two blocks unfrozen (the v1 setup), the model plateaued at 0.74 `val_acc` after 50 epochs. Unfreezing features.10–19 (ten blocks) and adding label smoothing 0.1 plus cosine annealing ($\text{lr } 1\text{e-}3 \rightarrow 1\text{e-}5$) pushed it to 0.77 `val_acc` with early stopping at epoch 41. Dropout 0.3 on the shared FC layer was a small improvement over 0.5 - the larger dataset made regularization less aggressive. Random erasing and 15° rotation in the augmentation pipeline helped with generalization to off-angle photos.

8. Results

Model	car_type	door_count	seat_count
Naive (majority)	0.229	0.577	0.576
Classical (RF)	0.407	0.611	0.609
Deep (MobileNetV2)	0.771	0.819	0.860

969 held-out real photos, stratified by car type. Top-5 accuracy is 1.000 across all tasks for the deep model - which is a vacuous number here since there are only 3–5 classes per task, so I’m not leaning on it.

The naive baseline tells you what “doing nothing” buys: 23% on car_type (majority class is “coupe” due to the slight imbalance), 58% on door and seat (because “5” is the majority class for both). Any model worth its parameters has to clear those.

Classical roughly doubles car_type accuracy (0.41 vs 0.23) and slightly beats naive on door and seat. The 50 handcrafted features carry real signal - body proportions, color, and texture do separate vehicle types - but they’re not enough to fully disambiguate visually similar classes (sedan vs hatchback, suv vs mpv).

The deep model wins on all three tasks by a wide margin. The 36-point lift on car_type (0.77 vs 0.41) is the cleanest evidence that learned features beat handcrafted ones on real photos. Door and seat benefit from the shared backbone: a coupe’s silhouette tells you both “2 doors” and “2 seats” at once, and the multi-task loss lets the model use that coupling. On the five live demo samples selected from the test set, the deep model predicts all five correctly with confidence 0.97–0.98.

Confusion matrices for every model × task pair are in data/outputs/plots/.

9. Error Analysis

The held-out test set produced 222 misclassifications on car_type out of 969 images. Five representative cases are listed below.

#	True	Predicted	Confidence	What went wrong
1	hatchback	sedan	0.82	Side-view hatchback and sedan are nearly indistinguishable at 224×224; the model commits strongly to the wrong class
2	coupe	sedan	0.80	Coupe silhouette is low and long, easy to confuse with a sedan at certain angles
3	sedan	mpv	0.73	Sedan shot from a high angle can look tall, triggering the MPV pattern
4	hatchback	coupe	0.84	Two-door hatchbacks share roofline with coupes; the model keys on roof height

#	True	Predicted	Confidence	What went wrong
5	suv	coupe	0.37	Low confidence - the model is genuinely uncertain, which the confidence gate would catch

Most errors are between two visually similar body types. The confidences vary widely (0.33–0.84), which means some errors are honest uncertainty (low confidence, gate-able) while others are committed mistakes (high confidence, harder to catch). The hatchback $\leftrightarrow$ sedan confusion dominates the error set, consistent with these two classes sharing the most visual overlap in side-view photos.

10. Experiments

Robustness under noise

I took the held-out test set and added gaussian noise at three severity levels ($\sigma = 0.05, 0.10, 0.20$), then re-ran the deep model.

Severity	Accuracy
0.05	0.705
0.10	0.579
0.20	0.346
Mean	0.543

The model drops from 0.77 clean to 0.35 at the heaviest noise, a 42-point drop. For comparison, the classical model barely moves under pixel noise because it operates on binned histograms and tree thresholds, not raw pixels. A production system that needs to survive bad lighting or motion blur could ensemble the two: deep when you can, classical when the input is dirty.

Confidence gating

The idea is to abstain on low-confidence predictions and only commit when the model is sure enough [3]. Threshold sweeps on the deep model:

Threshold	Coverage	Accuracy on predicted
0.2	1.000	0.771
0.3	0.996	0.774
0.4	0.960	0.797
0.5	0.890	0.824
0.6	0.814	0.858
0.7	0.755	0.884
0.8	0.648	0.925

Raise the threshold and coverage drops while accuracy on the committed predictions rises - the trade-off you'd expect. At threshold 0.5 the model keeps 89% coverage while pushing accuracy

to 0.82. At 0.8 it commits on 65% of cases and gets 93% of them right. For deployment, a threshold around 0.5 seems like a good operating point - you keep most of the coverage while gaining 5 points of accuracy, and the abstained cases get routed to a human reviewer or the classical fallback.

Head/tail class analysis

Per-class accuracy on car_type:

Class	Accuracy
sedan	0.654
suv	0.820
mpv	0.905
coupe	0.802
hatchback	0.672

The gap between best (mpv, 0.91) and worst (sedan, 0.65) is 25 percentage points. The head/tail gap is much smaller than the v1 run (which had a 100-point gap on 20 test images), which suggests the larger dataset smoothed out the per-class variance. Sedan is the hardest class - it sits visually between hatchback and coupe, and many sedans in the crawled set have ambiguous rooflines. MPV is the easiest, which makes sense given its distinct tall-box silhouette.

11. What I'd Do Differently

The biggest lever was data, and I already pulled it. Going from 100 photos to 4,869 pushed car_type accuracy from 0.55 to 0.77 - a 22-point gain that confirmed the v1 bottleneck was the dataset, not the architecture. The next step is to actually download CompCars (or Stanford Cars, or Cars196) and train on the full 136k-image set with a GPU. That alone should push car_type accuracy into the 0.85–0.90 range.

The deep model was trained on MPS in 80 epochs with early stopping at epoch 41, which took about 40 minutes. With a real GPU, 150–200 epochs and a larger backbone (ResNet50 or EfficientNet-B0) would be feasible. The classical model is already at its ceiling for 50 hand-crafted features - more trees or deeper trees won't help. What it needs is richer features (contour curvature, roofline angles, wheel-base ratio), not a bigger model.

Multi-view fusion is the other obvious win. CompCars has view annotations (front, rear, side, front-side, rear-side), and a side view is much better for door count than a front view. Currently the model treats all views the same.

Grad-CAM would be a nice addition to the explanation layer - right now the user sees feature narratives ("dominant color is red"), but a heatmap showing *where* the CNN is looking would make the explanation visual, not just textual.

12. Conclusion

I ended up with one model that predicts three correlated attributes from a single photo, with a human-readable explanation layer attached. The deep model beats both baselines on all three

tasks by a wide margin (77% car_type, 82% door, 86% seat). The classical model proves the handcrafted features carry real signal. And the web app lets you drag a real photo in and see the result in under a second.

The numbers are honest - 77% car_type accuracy on a 969-image test set is a real result, not a cherry-picked one. The architecture works - multi-task learning helps, the shared backbone picks up attribute correlations, and the interpretability layer runs on real images. Scaling up to the full CompCars dataset is the obvious next step, and the pipeline is ready for it.

13. Future Work

A few directions worth pursuing if I had more time: train on full CompCars (136k images) with a GPU; add Grad-CAM heatmaps to the explanation layer; try SE-Block or CBAM attention modules on the backbone; fuse multiple views using CompCars' view annotations (front, rear, side, front-side, rear-side); export to TensorRT or ONNX for sub-100ms inference; and build a mobile client (React Native or Flutter) so the demo isn't browser-only.

14. Commercial Viability

The realistic use cases are used-car valuation platforms (auto-fill vehicle attributes from a listing photo), fleet management (quick inventory entry), and insurance (verification of declared vehicle type). The multi-task setup is a real cost saver - one model instead of three - and the explanation layer helps with user trust, which matters in regulated contexts.

But 77% car_type accuracy isn't good enough for any of these yet. The architecture is ready; the data is close but not quite there. With a GPU and the full CompCars dataset, the same codebase should reach deployable accuracy in a few days of training. After that, the work is integration: real-world lighting and angle variation, latency budgets, and a feedback loop for edge cases.

15. Ethics

CompCars is non-commercial research use only; the real photos used for training were crawled from Bing Image Search via icrawler for research purposes. No license plates or personally identifiable information is stored - uploaded images are processed in memory and discarded.

The main bias risk is class imbalance in the training data: if certain vehicle types are underrepresented, the model will be worse for owners of those types, which could matter if the system is used for pricing. The `class_weight="balanced"` setting and the head/tail analysis in §10 are the current mitigations; on a larger dataset, oversampling minority classes or using focal loss would be the next step.

The explanation layer also helps on the ethics side: a user who gets a wrong prediction can at least see *why* the model was confused ("dominant color is red, aspect ratio 1.1 - leans sedan"), which is better than a bare label with no context.

Repository Layout

— main.py	Flask app
— setup.py	full training pipeline (synthetic)
— scripts/	

		data.py	loading + class mappings
		features.py	50-D handcrafted features
		model.py	Naive / Classical / Deep
		experiment.py	experiment harness
		synthetic_data.py	synthetic generator
		download_large_dataset.py	Bing image crawler (icrawler)
		clean_dataset.py	dedup + validation + aspect filter
		train_large.py	train on 4869 real images (MPS)
		eval_large.py	re-evaluate + regenerate metrics.json
		update_demo_samples.py	pick correct test-set samples for demo
		eda.py	exploratory analysis
		make_pptx.py	slide generator
		models/	7 trained files (on HF Hub)
		data/	
			real_cars_large/ 4869 cleaned real photos
			outputs/ metrics + plots
		static/	UI assets + 5 demo samples
		templates/	index.html
		Dockerfile	

Trained model files: models/naive_*.pkl (Naive baseline), models/classical_*.pkl (Classical RF), models/deep_multitask.pt (Deep MobileNetV2). Each maps to the corresponding class in scripts/model.py.

References

- [1] L. Yang, P. Luo, C. C. Loy, and X. Tang, “A large-scale car dataset for fine-grained categorization and verification,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 1597–1605.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510–4520.
- [3] Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4878–4887.
- [4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.